



Green University of Bangladesh

Department of Computer Science and Engineering (CSE)

Faculty of Sciences and Engineering

Semester: (Summer, Year: 2026), B.Sc. in CSE (Day)

LAB REPORT NO - 02

Course Title: Computer Networking Lab

Course Code: CSE_318 Section: 241-D2

**Lab Experiment Name: Multithreaded Socket Programming
Using Java**

Student Details

Name		ID
1.	MD.SHAJALAL	223002088

Submission Date : 02 – 08 - 2026

Course Teacher's Name : Md. Mahbubur Rahman

Lab Report Status

Marks:

Signature:.....

Comments:.....

Date:.....

1.Objective

- To design and implement a client-server messaging system using Java socket programming.
- To develop a multithreaded server that can handle multiple clients simultaneously.
- To create a university help desk communication model using client-server architecture.
- To use Java threads so each client session runs independently.
- To ensure that one client's activity does not block or interrupt other connected clients.
- To understand the concepts of socket communication, multithreading, and concurrent network programming in Java.

2.Theory and Background

Java socket programming provides the `ServerSocket` and `Socket` classes for establishing TCP connections between a server and clients. A basic single-threaded server can only serve one client at a time — while it is reading from or writing to one client's socket, all other clients waiting to connect or communicate are blocked.

To overcome this limitation, the server is made multithreaded: every time a new client connects, the server spawns a new thread (or `Runnable` task) dedicated to that client. This thread independently manages the client's input/output streams using `BufferedReader` (for reading) and `PrintWriter` (for writing). Because each client is served by its own thread, all clients can send and receive messages concurrently, and the server remains responsive to new incoming connections at all times.

3. System Design

The system has two Java programs:

- **HelpDeskServer.java** – This program starts the server and waits for students to connect on **port 5000**. Whenever a new student connects, the server creates a separate thread to handle that student. It asks for the student's name, sends a welcome message, receives messages from the student, and replies to each one. When the student sends "**TATA**", the server sends a goodbye message and closes only that student's connection.

- **HelpDeskClient.java** – This program connects to the server. It sends the student's name, allows the user to type messages from the keyboard, and displays the server's replies. When the user types "TATA", the client closes the connection.

Key Java classes used:

- **ServerSocket** – Creates the server and waits for client connections.
- **Socket** – Connects the client and server.
- **Thread / Runnable** – Allows multiple clients to communicate with the server at the same time.
- **BufferedReader** – Reads messages from the client and server.
- **PrintWriter** – Sends messages between the client and server.
- **ConcurrentHashMap-backed Set** – Keeps track of the names of clients currently connected to the server.

4. Source Code

4.1 HelpDeskServer.java

```
import java.io.*;
import java.net.*;
import java.util.*;
import java.util.concurrent.*;

public class HelpDeskServer {

    private static final int PORT = 5000;

    private static final Set<String> connectedClients = ConcurrentHashMap.newKeySet();

    public static void main(String[] args) {

        System.out.println("Server started on port " + PORT + "...");

        try (ServerSocket serverSocket = new ServerSocket(PORT)) {

            int clientCount = 0;

            while (true) {

                Socket clientSocket = serverSocket.accept();
```

```

        clientCount++;

        String defaultTag = "Client-" + clientCount;

        ClientHandler handler = new ClientHandler(clientSocket, defaultTag);

        Thread t = new Thread(handler);

        t.start();
    }

} catch (IOException e) {

    System.out.println("Server exception: " + e.getMessage());

}

}

static class ClientHandler implements Runnable {

    private final Socket socket;

    private final String connectionTag;

    private String clientName = "Unknown";

    public ClientHandler(Socket socket, String connectionTag) {

        this.socket = socket;

        this.connectionTag = connectionTag;

    }

    @Override

    public void run() {

        try (

            BufferedReader in = new BufferedReader(

                new InputStreamReader(socket.getInputStream()));

            PrintWriter out = new PrintWriter(socket.getOutputStream(), true)

        ) {

            System.out.println("Client connected: " + connectionTag);

            out.println("Enter your name:");

            String name = in.readLine();

```

```

        if (name == null || name.trim().isEmpty()) {
            name = connectionTag;
        }

        clientName = name.trim();

        connectedClients.add(clientName);

        out.println("Welcome " + clientName + "! You are connected to the university help
desk.");

        String message;

        while ((message = in.readLine()) != null) {

            if (message.equalsIgnoreCase("TATA")) {

                out.println("Goodbye " + clientName + ". Your connection has been closed.");

                break;

            }

            System.out.println(clientName + ": " + message);

            out.println("Message received from " + clientName + ": " + message);

        }

    } catch (IOException e) {

        System.out.println(clientName + " connection error: " + e.getMessage());

    } finally {

        connectedClients.remove(clientName);

        try {

            socket.close();

        } catch (IOException e) {

        }

        System.out.println(clientName + " disconnected.");

        System.out.println("Server is still serving: " +

            (connectedClients.isEmpty() ? "no other clients" : connectedClients));

    }

}

```

```
}  
}
```

4.2 HelpDeskClient.java

```
import java.io.*;  
  
import java.net.*;  
  
public class HelpDeskClient {  
  
    private static final String SERVER_ADDRESS = "localhost";  
  
    private static final int SERVER_PORT = 5000;  
  
    public static void main(String[] args) {  
  
        try (  
  
            Socket socket = new Socket(SERVER_ADDRESS, SERVER_PORT);  
  
            BufferedReader in = new BufferedReader(  
  
                new InputStreamReader(socket.getInputStream()));  
  
            PrintWriter out = new PrintWriter(socket.getOutputStream(), true);  
  
            BufferedReader keyboard = new BufferedReader(new InputStreamReader(System.in))  
  
        ) {  
  
            System.out.println(in.readLine());  
  
            String name = keyboard.readLine();  
  
            out.println(name);  
  
            System.out.println("Server: " + in.readLine());  
  
            String userInput;  
  
            while (true) {  
  
                System.out.print(name + ": ");  
  
                userInput = keyboard.readLine();  
  
                if (userInput == null) break;  
  
                out.println(userInput);  
  
            }  
  
        }  
  
    }  
}
```

```

        String response = in.readLine();

        if (response == null) break;

        System.out.println("Server: " + response);

        if (userInput.equalsIgnoreCase("TATA")) {
            break;
        }
    }

    System.out.println("Disconnected from the server.");
} catch (IOException e) {
    System.out.println("Client exception: " + e.getMessage());
}
}
}

```

5. Output

5.1 Console

```

Server started on port 5000...
Client connected: Client-1
Client connected: Client-2
Client connected: Client-3
Mahbub: Hello Server
Rahim: I need information about lab classes.
Karim: What is the submission deadline?
Mahbub: Is the lab open today?
Rahim disconnected.

```

```
Server is still serving: [Mahbub, Karim]
Karim: Can I get an extension?
Mahbub disconnected.
Server is still serving: [Karim]
Karim disconnected.
Server is still serving: no other clients
```

5.2 Client 1 Console — "Mahbub"

```
Enter your name:
Server: Welcome Mahbub! You are connected to the university help desk.
Mahbub: Server: Message received from Mahbub: Hello Server
Mahbub: Server: Message received from Mahbub: Is the lab open today?
Mahbub: Server: Goodbye Mahbub. Your connection has been closed.
Disconnected from the server.
```

5.3 Client 2 Console — "Rahim"

```
Enter your name:
Server: Welcome Rahim! You are connected to the university help desk.
Rahim: Server: Message received from Rahim: I need information about lab classes.
Rahim: Server: Goodbye Rahim. Your connection has been closed.
Disconnected from the server.
```

5.4 Client 3 Console — "Karim"

```
Enter your name:
Server: Welcome Karim! You are connected to the university help desk.
Karim: Server: Message received from Karim: What is the submission deadline?
Karim: Server: Message received from Karim: Can I get an extension?
Karim: Server: Goodbye Karim. Your connection has been closed.
Disconnected from the server.
```

6. How Multithreading Allows the Server to Serve Multiple Clients

In this system, the server only waits for new client connections. When a new client connects, the server creates a **new thread** to handle that client. This allows many clients to use the server at the same time.

- **Accepts new clients continuously:** The server keeps waiting for new connections while other threads handle existing clients.
- **Each client has its own communication:** Every client gets a separate thread, so one client's messages do not affect another client.
- **Independent connections:** If one client disconnects by typing "**TATA**", only that client's connection is closed. Other clients can continue chatting with the server.
- **Safe client management:** The server keeps a thread-safe list of connected clients, so multiple threads can add or remove client names without causing errors.
- **Supports many users:** Because each client runs in its own thread, many students can communicate with the server at the same time.

7. Conclusion

This lab successfully created a multithreaded client-server messaging system using Java socket programming. The server can handle multiple clients at the same time by giving each client its own thread. Each client can send and receive messages independently without affecting others. When a client types "**TATA**", only that client's connection is closed, while the server and other clients continue working normally. The results show that at least three clients can communicate with the server simultaneously.